

Phase 1 Test Cases

Login Test Cases

Test Case #	Input	Expected Output
1. User enters an existing admin username	Username: AdminUser	Status: Accepted. Reason: Existing admin user successfully logged in
2. User enters an existing full-standard username	Username: FullUser	Status: Accepted. Reason: Existing user successfully logged in
3. User enters an existing buy-standard username	Username: BuyUser	Status: Accepted. Reason: Existing user successfully logged in
4. User enters an existing sell-standard username	Username: SellUser	Status: Accepted. Reason: Existing user successfully logged in
5. User enters a username that does not exist	Username: InvalidUser	Status: Denied. Reason: Username does not exist
6. User attempts another transaction before logging in	User tries to buy a game before login	Status: Denied. Reason: User must login before performing another transaction
7. User attempts to login again while already logged in	User logs in successfully, then enters login again	Status: Denied. Reason: User is already logged in
8. User logs in after previously logging out	Login -> Logout -> Login	Status: Accepted. Reason: Login is permitted after previous session has ended
9. User leaves username blank	Username: [blank]	Status: Denied. Reason: Blank input is handled gracefully and the Front End continues

		accepting input.
--	--	------------------

Logout Test Cases

1. User logs out while currently logged in	Logged in user enters logout	Status: Accepted. Reason: Logout successful; Daily Transaction File is written
2. User tries to logout without being logged in	Logout before login	Status: Denied. Reason: User must be logged in to logout
3. User tries to buy after logging out	Login -> Logout -> Buy	Status: Denied. Reason: User must login before buying
4. User tries to sell after logging out	Login -> Logout -> Sell	Status: Denied. Reason: User must login before selling
5. User tries to create after logging out	Login -> Logout -> Create	Status: Denied. Reason: User must login first
6. User tries to delete after logging out	Login -> Logout -> Delete	Status: Denied. Reason: User must login first
7. User tries to add credit after logging out	Login -> Logout -> Add Credit	Status: Denied. Reason: User must login first
8. User tries to refund after logging out	Login -> Logout -> Refund	Status: Denied. Reason: User must login first
9. User logs in again after logout	Login -> Logout -> Login	Status: Accepted. Reason: New session may begin
10. User tries to list games after logging out	Login -> Logout -> list-games transaction	Status: Denied. Reason: User must login first

Create Test Cases

1. Admin creates an admin account	Username: NewAdmin; Type: Admin	Status: Accepted. Reason: New admin account created
2. Admin creates a full-standard account	Username: NewFull; Type:full-standard	Status: Accepted
3. Admin creates a buy-standard account	Username: NewBuyer; Type: buy-standard	Status: Accepted
4. Admin creates a sell-standard account	Username: NewSeller; Type: sell-standard	Status: Accepted
5. Non-admin user attempts to create a user	Full-standard user enters create	Status: Denied. Reason: Create is an admin-only transaction
6. Admin tries to create a duplicate username	New username:ExistingUser	Status: Denied. Reason: Username already exists
7. Admin creates a username exactly 15 characters long	Username: ABCDEFGHIJKLMNO	Status: Accepted. Reason: Username is within maximum length
8. Admin tries username longer than 15 characters	Username: ABCDEFGHIJKLMNOP	Status: Denied. Reason: Username exceeds 15-character maximum
9. Admin enters an invalid user type	Username: NewUser; Type: randomtype	Status: Denied. Reason: Invalid user type
10. Admin leaves username blank	Username: [blank]	Status: Denied. Reason: Blank username input is handled gracefully
11. Newly created account starts with 0 credit	Admin creates a valid new user	Status: Accepted. New account starts with a credit balance of 0.00.

Delete Test Cases

1. Admin deletes an existing user	Username: David	Status: Accepted. Reason: User account deleted
2. Non-admin tries to delete a user	Full-standard user enters delete	Status: Denied. Reason: Delete is admin-only
3. Admin tries to delete a user that does not exist	Username: DavidZ	Status: Denied. Reason: User does not exist
4. Admin tries to delete their own currently logged-in account	Username: AdminUser	Status: Denied. Reason: Current user cannot delete themselves
5. Transaction attempts to use deleted user's game inventory	Delete seller -> attempt transaction on seller's game	Status: Denied. Reason: Deleted user's inventory is no longer available
6. Admin enters blank username	Username: [blank]	Status: Denied. Reason: Blank username input is handled gracefully
7. Deleting a user removes their Game Collection entries	Admin deletes an existing user who owns games in the Game Collection File	Status: Accepted. Game Collection File entries associated with the deleted user are removed.

Sell Test Cases

1. Full-standard user sells a valid game	Game: Minecraft; Price: 25.00	Status: Accepted. Reason: Game successfully listed
2. Sell-standard user sells a valid game	Game: Terraria; Price: 15.00	Status: Accepted
3. Admin sells a valid	Game: Portal; Price:	Status: Accepted

game	20.00	
4. Buy-standard user attempts to sell	Buy-standard user enters sell	Status: Denied. Reason: Buy-standard users cannot sell
5. User lists game for exactly \$999.99	Price: 999.99	Status: Accepted
6. User lists game for more than \$999.99	Price: 1000.00	Status: Denied. Reason: Maximum game price is \$999.99
7. User enters a game name exactly 26 characters long	26-character game name	Status: Accepted. Reason: Game name matches the example defined maximum field length.
8. User enters a game name longer than 26 characters	27-character game name	Status: Denied. Reason: Game name exceeds the example-defined maximum field length.
9. User tries to sell a game whose name already exists	Game: existing game name	Status: Denied. Reason: Game names must be unique
10. User tries to use newly listed game during the same session	Sell new game -> attempt buy/action on it	Status: Denied. Reason: New game cannot be used until next session
11. User enters letters instead of a price	Price: abc	Status: Denied. Reason: Invalid price
12. User leaves game name blank	Game: [blank]	Status: Denied. Reason: Blank game-name input is handled gracefully.
13. User enters a price of 0.00	Price: 0.00	Status: Accepted. Reason: Price is within the allowed range of \$0.00 to \$999.99.
14. User enters a	Price: -5.00	Status: Denied. Reason:

negative price		Price is outside the allowed range of \$0.00 to \$999.99.
15. Newly listed game is used in the next session	Session 1: Sell game -> Logout. Session 2: attempt valid transaction on that game	Status: Accepted if all other transaction requirements are satisfied

Buy Test Cases

1. Full-standard user purchases an existing game	Game: Minecraft; Seller: Seller1	Status: Accepted. Reason: Purchase successful
2. Buy-standard user purchases a game	Game: Portal; Seller: Seller2	Status: Accepted
3. Admin purchases a game	Game: Terraria; Seller: Seller3	Status: Accepted
4. Sell-standard user tries to buy	Sell-standard user enters buy	Status: Denied. Reason: Sell-standard users cannot buy
5. User attempts to buy nonexistent game	Game: FakeGame	Status: Denied. Reason: Game does not exist
6. Buyer does not have enough credit	Game price: 50.00; Buyer credit: 20.00	Status: Denied. Reason: Insufficient credit
7. Buyer has exactly enough credit	Game price: 50.00; Buyer credit: 50.00	Status: Accepted
8. Buyer already owns game	Buyer attempts to purchase owned game	Status: Denied. Reason: Buyer already owns this game

9. User leaves game name blank	Game: [blank]	Status: Denied. Reason: Blank game-name input is handled gracefully.
10. User enters wrong seller for existing game	Correct game; incorrect seller username	Status: Denied. Reason: Seller does not match the seller associated with that game.
11. Successful purchase updates buyer and seller credit	Buyer purchases game for \$50.00	Status: Accepted. Buyer balance decreases by \$50.00 and seller balance increases by \$50.00
12. Successful purchase adds game to buyer collection	Buyer purchases valid game	Status: Accepted. Purchased game is added to buyer's collection
13. Seller attempts to buy their own listed game	Game: Minecraft; Seller: Seller1; logged-in user: Seller1	Status: Denied. Reason: Seller cannot buy their own game

Refund Test Cases

1 Admin performs valid refund	Buyer: Buyer1; Seller: Seller1; Amount: 20.00	Status: Accepted. Reason: Refund transferred
2 Non-admin attempts refund	Full-standard user enters refund	Status: Denied. Reason: Refund is admin-only
3 Buyer username does not exist	Buyer: InvalidBuyer; Seller: Seller1	Status: Denied. Reason: Buyer does not exist
4 Seller username does not exist	Buyer: Buyer1; Seller: InvalidSeller	Status: Denied. Reason: Seller does not exist
5 Neither buyer nor	InvalidBuyer;	Status: Denied

seller exists	InvalidSeller	
6 Buyer field is blank	Buyer: [blank]	Status: Denied. Reason: Invalid input
7 Seller field is blank	Seller: [blank]	Status: Denied. Reason: Invalid input
8 Refund amount is not numeric	Amount: abc	Status: Denied. Reason: Invalid refund amount
9. Valid refund updates both account balances	Buyer: Buyer1; Seller: Seller1; Amount 20.00	Status: Accepted. Seller credit decreases by 20.00 and buyer credit increases by 20.00
10. Seller has exactly enough credit for refund	Seller credit: 20.00 Refund: 20.00	Status: Accepted. Seller balance becomes 0.00; buyer receives 20.00
11. Seller does not have enough credit	Seller credit: 10.00 Refund: 20.00	Status: Denied. Reason: Seller has insufficient credit
12. Refund amount is 0.00	Amount: 0.00	Status: Denied. Reason: Refund amount must be greater than 0.00
13. Refund amount is negative	Amount: -20.00	Status: Denied. Reason: Refund amount must be positive
14. Successful refund removes game from buyer's collection	Buyer: Buyer1; Seller: Seller1; valid refund for a game previously purchased by Buyer1	Status: Accepted. Refunded game is removed from the buyer's Game Collection File
15. Buyer repurchases a previously refunded game	Session 1: successful refund -> logout. Session 2: buyer attempts to purchase the refunded game again	Status: Accepted if all normal buy requirements are satisfied. A previously refunded game may be purchased again.

Add Credit Test Cases

1. Standard user adds valid amount to account	Amount: 100.00	Status: Accepted
2. Admin adds credit to existing user	Amount 100.00; Username: User1	Status: Accepted
3. Admin tries to add credit to nonexistent user	Username: InvalidUser	Status: Denied. Reason: User does not exist
4. User adds exactly \$1000 in one session	Amount: 1000.00	Status: Accepted
5. User tries to add more than \$1000	Amount: 1000.01	Status: Denied. Reason: Session credit limit exceeded
6. User makes multiple additions totaling exactly \$1000	Amounts: 400.00 + 600.00	Status: Accepted
7. User makes multiple additions totaling over \$1000	Amounts: 600.00 + 500.00	First addition: Accepted. Second addition: Denied. Reason: Cumulative credit added during the session would exceed \$1000.00.
8. User enters nonnumeric amount	Amount: abc	Status: Denied. Reason: Invalid credit amount
9. User leaves amount blank	Amount: [blank]	Status: Denied. Reason: Invalid input
10. Successful addcredit updates account balance	Existing balance: 100.00; Amount 50.00	Status: Accepted. Account credit increases to 150.00
11. Addcredit would exceed maximum account balance	Existing balance: 999500.00; Amount: 600.00	Status: Denied. Reason: Account credit would exceed the maximum allowed balance of

	999500.00 + 600.00 = 1000100.00 > 999999.99	999999.99
--	--	-----------

List Games Test Cases

1. User requests list when several games are available	list_games	Status: Accepted. All available games displayed
2. Only one game is available	list_games	Displays one game
3. No games are available	list_games	Displays message that no games are currently available
4. Verify game names	list_games	All displayed game names match Available Games File
5. Verify seller usernames	list_games	Correct seller displayed for each game
6. Verify prices	list_games	Correct price displayed for each game
7. User tries list before login	list_games before login	Status: Denied. Reason: User must login before listing available games.
8. User tries list after logout	Logout -> list_games	Status: Denied. Reason: User must login before listing available games.
9. Admin user uses list_games	Admin user logs in and enters list_games	Status: Accepted. All relevant information for currently available games is displayed.
10. Full-standard user uses list_games	Full-standard user logs in and enters list_games	Status: Accepted. All relevant information for

		currently available games is displayed.
11. Buy-standard user uses list_games	Buy-standard user logs in and enters list_games	Status: Accepted. All relevant information for currently available games is displayed.
12. Sell-standard user uses list_games	Sell-standard user logs in and enters list_games	Status: Accepted. All relevant information for currently available games is displayed.

General Invalid Input Test Cases

1. User enters invalid transaction name	banana	Status: Denied. Reason: Invalid transaction; program continues
2. User enters blank transaction	[blank line]	Status: Denied. Program continues
3. User enters letters for game price	Price: abc	Invalid price handled gracefully
4. User enters letters for credit amount	Credit: abc	Invalid amount handled gracefully
5. User enters letters for refund amount	Refund: abc	Invalid amount handled gracefully
6. User enters excessive text	Extremely long field	Input rejected gracefully
7. User enters valid transaction after invalid one	Banana -> login	Invalid input rejected; login still works
8. Invalid input followed by another valid transaction in same	Invalid input -> valid transaction	Invalid input is rejected; Front End continues accepting valid

session		transactions
9. Very long transaction command	Extremely long command string	Invalid input handled gracefully; program does not crash
10. Special characters used as transaction input	@#\$%^	Invalid transaction handled gracefully; program continues

Transaction Output / Daily Transaction File Cases

1. Valid create transaction	Create user -> logout	Daily transaction File contains correct 01 record, then 00
2. Valid delete transaction	Delete user -> logout	Correct 02 record, then 00
3. Valid sell transaction	Sell game -> logout	Correct 03 record, then 00
4. Valid buy transaction	Buy game -> logout	Correct 04 record, then 00
5. Valid refund transaction	Refund -> logout	Correct 05 record, then 00
6. Valid addcredit transaction	Addcredit -> logout	Correct 06 record, then 00
7. Session with several valid transactions	Perform several transactions -> logout	All records appear in correct order, followed by 00
8. Failed transaction is omitted from Daily Transaction File	Login -> attempt to buy a FakeGame -> logout	Failed transaction does not create a Daily Transaction File record. Only successful transactions are recorded, followed by 00.

9. Verify username field length and alphabetic alignment	Successful transaction using a short username	Username field is 15 characters long, left-justified, and padded with spaces in unused positions.
10. Verify monetary formatting	Transaction using whole-dollar amount	Monetary field includes .00
11. Verify end-of-session record	Logout	Final transaction code is 00
12. Verify unused numeric fields	Transaction containing unused numeric field	Unused numeric fields are filled with zeros
13. Verify unused alphabetic fields	Transaction containing unused alphabetic field	Unused alphabetic fields are filled with spaces
14. Verify sell record field order	Successful sell -> logout	03 record contains game name, seller username, and price in the required order
15. Verify buy record field order	Successful buy -> logout	04 record contains game name, seller username, buyer username, and price in required order
16. Verify refund record field order	Successful refund -> logout	05 record contains buyer username, seller username, and refund amount in required order
17. Verify create/delete/addcredit record field order	Successful create/delete/addcredit -> logout	Record contains transaction code, username, user type, and credit fields in required order
18. Verify all valid session transactions are recorded exactly once	Several successful transactions → logout	Every successful transaction appears once, in session order, followed by 00

